Exact Implementation Specification: NF-iSAM

Faithful implementation plan for "Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows"

0. Purpose and fidelity rules

This document specifies how to implement NF-iSAM as described by Huang et al. It focuses on Bayes-tree clique conditional modeling using normalizing flows, training-sample simulation, conditional sampler extraction, and incremental updates. The goal is to reproduce the authors' algorithmic pipeline, not to optimize or replace it.

Fidelity rule

Algorithms 1-3 are implemented as the authoritative flow. Normalizing-flow parameterization follows the authors' current implementation: autoregressive rational-quadratic spline flows with fully connected conditioner networks. When code-level details are not specified, keep configurable defaults and mark the choice.

Source anchor: The NF-iSAM paper states that it trains normalizing flows to model/sample the full posterior and leverages the Bayes tree for efficient incremental updates; see abstract and Sections III-IV.

1. Inputs, outputs, and core objects

Object	Meaning	Implementation structure
G(f, Θ, E)	Factor graph with variables Θ and factors f.	FactorGraph class.
C	Clique on Bayes tree; also denotes variables in clique.	BayesTreeClique object.
F_C	Frontal variables of clique C.	Ordered variable subset in clique.
S_C	Separator variables: intersection of clique C and its parent.	Ordered variable subset; may be empty at root.
z_C	Observations in and below clique C.	Clique-local measurement/factor scope.
p(F_C | S_C, z_C)	Clique conditional to learn with normalizing flows.	ConditionalSampler object wrapping flow blocks.
p(S_C | z_C)	Separator density passed upward as a new factor.	SeparatorDensity object wrapping T_S.
O_C	Unobserved measurement variables introduced for selected loop-closing factors in intermediate sampling.	ObservationVariable block in training samples.
T_C	Desired normalizing flow for clique density p(S_C,F_C|z_C).	Triangular autoregressive flow partitioned into T_S and T_F.
T̃_C	Intermediate normalizing flow trained on samples of (O_C,S_C,F_C).	Training flow before fixing observations.

2. Bayes-tree factorization and inference task

$$\text{Eq. N1: } p(\Theta \mid z) = \prod_{C \in \mathcal{C}} p(F_C \mid S_C, z) = \prod_{C \in \mathcal{C}} p(F_C \mid S_C, z_C)$$

The implementation must convert the factor graph to a Bayes tree using a variable elimination ordering. The paper uses the Bayes tree because it decomposes the high-dimensional posterior into a sequence of lower-dimensional clique conditionals. Each clique conditional is learned by a normalizing flow and later sampled during a root-to-leaf traversal.

$$\text{Eq. N2: } p(F_C, S_C \mid z_C) \propto \left[\prod_{q \in C_C} p(S_q \mid z_q)\right] \left[\prod_{\Theta_{f_i} \subseteq C,\ \Theta_{f_i} \not\subset S_q} f_i(\Theta_{f_i})\right]$$

Source anchor: Bayes-tree clique conditional factorization and clique density are given in paper Eq. (2)-(3), Section III; p. 4.

3. Normalizing-flow model

$$\text{Eq. N3: } T(x) = [T_1(x_1),\ T_2(x_1,x_2),\ \ldots,\ T_D(x_1,\ldots,x_D)]^T$$

$$\text{Eq. N4: } p(x;T) = q(T(x)) \prod_{d=1}^{D} \partial T_d / \partial x_d$$

$$\text{Eq. N5: } p(x_d \mid x_1,\ldots,x_{d-1};T) = q(y_d)\, \partial T_d / \partial x_d,\ \ y_d = T_d(x_1,\ldots,x_d)$$

$$\text{Eq. N6: sample } y \sim q(y)=\mathcal{N}(0,I_D), \text{ then } x = T^{-1}(y) \text{ by forward substitution}$$

$$\text{Eq. N7: } T^* = \operatorname{argmin}_{T\in\mathbb{F}} \sum_{k=1}^{n} \sum_{d=1}^{D} \left[\frac{1}{2} T_d^2(x^{(k)}) - \log(\partial T_d/\partial x_d)(x^{(k)}) \right]$$

Source anchor: Triangular map, density evaluation, conditional extraction, sampling, and training objective are from paper Eq. (4)-(17), Section IV-A; pp. 5-6.

4. Modeling a clique conditional: two-step strategy

NF-iSAM does not directly train on p(S_C,F_C|z_C) by slow MCMC or nested sampling. Instead, it simulates samples from an efficiently sampleable intermediate density, trains $\tilde{T}$_C on those samples, fixes observed measurements, and extracts the desired T_C.

$$\text{Eq. N8: } T_C(S_C,F_C) = \left[T_{S_C}(S_C),\ T_{F_C}(S_C,F_C) \right]^T$$

$$\text{Eq. N9: } \tilde{T}_C(O_C,S_C,F_C) = \left[\tilde{T}_{O_C}(O_C),\ \tilde{T}_{S_C}(O_C,S_C),\ \tilde{T}_{F_C}(O_C,S_C,F_C) \right]^T$$

$$\text{Eq. N10: } T_C(S_C,F_C) = \left[\tilde{T}_{S_C}(O_C=o_C,S_C),\ \tilde{T}_{F_C}(O_C=o_C,S_C,F_C) \right]^T$$

Step	Authors' operation	Implementation
Step 1	Draw training samples from intermediate density p(O_C,S_C,F_C | z_C*) where sampling is efficient.	Use TrainingSampleSimulator. Select loop-closing factors, turn measurement variables into unobserved variables O_C, and perform ancestral simulation.
Step 2	Train $\tilde{T}$_C using samples from Step 1, then fix O_C=o_C to retrieve the desired T_C.	Use ConditionalSamplerTrainer; partition T_C into T_S and T_F; obtain separator density sampler and clique conditional sampler.

Source anchor: Two-step strategy, intermediate density, loop-closing factors, and Eqs. (18)-(20) are given in Section IV-B; pp. 7-8.

5. Algorithm 1: TrainingSampleSimulator

```
Algorithm N1: TrainingSampleSimulator(P, B, M)
Input: prior factors P, binary measurement factors B, multi-modal data association factors M in a clique
Output: sample dictionary S and measured-value dictionary V

1. Initialize S and V as empty dictionaries.
2. Draw samples for any variable Θ_j in prior factors P; store in S[Θ_j].
3. while queue B is not empty:
       pop first binary factor f_i = p(z_i | Θ_j, Θ_k)
       if exactly one latent variable is not yet in S:
           simulate that missing variable using the known variable sample, measured z_i, and measurement
model; store in S.
       else if both latent variables are already in S:
           simulate virtual measurement Z_i between S[Θ_j] and S[Θ_k]; store in S[Z_i].
           store measured value z_i in V[Z_i].
       else:
           push f_i to the back of B.
4. for every multi-modal data-association factor f_i = p(z_i | Θ_j, Θ_k) in M:
       simulate virtual measurement Z_i between S[Θ_j] and S[Θ_k]; store in S[Z_i].
       store measured value z_i in V[Z_i].
5. return S,V.
```

Source anchor: Algorithm 1 appears on paper p. 8; the surrounding text explains priors, binary factors, passively/proactively identified loop-closing factors, and virtual observation simulation; pp. 7-8.

6. Algorithm 2: ConditionalSamplerTrainer

```
Algorithm N2: ConditionalSamplerTrainer(trainingSamples, measuredValues o)
Input: training samples and measured values o
Output: sampler for p(F|S) and sampler/density for p(S)

1. Rearrange training samples to the order: observations O, separator S, frontal variables F.
2. Train T̃ in Eq. N9 by minimizing the KL objective Eq. N7 using the training samples.
3. Fix observations: T(S,F) ← T̃(O=o, S, F).
```

```
4. Partition T(S,F) following Eq. N8 into T_S and T_F.
5. Obtain samplers of p(F|S) and p(S) from T_S and T_F by the inverse sampling procedure Eq. N6.
6. Return samplers p(F|S), p(S).
```

Source anchor: Algorithm 2 and the observation-fixing operation appear on paper p. 8; Eq. (20) gives the retrieved desired flow.

7. Algorithm 3: NF-iSAM incremental inference

```
Algorithm N3: NF-iSAM(newFactors f, factorGraph G, ordering Θ)
Input: new factors f, current factor graph G, variable ordering Θ
Output: samples D from the joint posterior distribution

1. G.update(f, Θ).
2. T_Δ ← T.extract(f, Θ)       // extract changed subtree of the Bayes tree.
3. for clique C in leaf-to-root traversal of T_Δ:
       x, o ← TrainingSampleSimulator(C)
       p(F_C|S_C), p(S_C) ← ConditionalSamplerTrainer(x, o)
       append p(S_C) to the parent clique as a factor.
4. D ← empty dictionary for posterior samples.
5. for clique C in root-to-leaf traversal of full Bayes tree T:
       retrieve stored conditional sampler p(F_C|S_C) from clique C.
       s ← D[S_C]                        // separator samples already generated by parent traversal.
       D[F_C] ← draw samples from p(F_C|S_C=s) by inverse flow sampling.
6. return D.
```

Source anchor: Algorithm 3 and the description of changed subtree training plus full root-to-leaf sampling appear on paper p. 8.

8. Incremental update policy

Incremental element	Exact paper behavior	Implementation rule
New factor arrival	The Bayes-tree change is exact and symbolic from the Bayes tree algorithm.	Call BayesTree.update/affectedSubtree to identify changed subtree T_Δ.
Training	Only cliques in the changed subtree are retrained.	Run Algorithm N1/N2 only for T_Δ cliques.
Reuse	Flows outside the subtree are unchanged and reused directly.	Do not retrain unaffected clique samplers.
Upward traversal	Starts from leaves of the changed subtree rather than the entire tree.	Traverse T_Δ leaf-to-root.
Downward traversal	Still visits all cliques to draw joint posterior samples.	Traverse full tree root-to-leaf after retraining; cost is lower than training.

Source anchor: Incremental update description appears after Algorithm 2: only changed subtree flows are learned, outside flows reused, downward traversal still visits all cliques; p. 8.

9. Current implementation settings from the authors

Setting	Author-specified value / behavior	Implementation
Programming language	Algorithms 1-3 and building blocks implemented in Python.	Python package with modules for factors, graph, Bayes tree, flows, training.
Flow type	Rational-quadratic splines for 1-D T_d(x_d; c_d(x_<d;w_d)).	Autoregressive RQ spline flow.
Conditioner network	Fully connected neural network with input dimension d-1.	FCNN per dimension d.
Spline parameter count	If K RQ functions, conditioner output dimension is 3K-1.	2K-2 knot-coordinate outputs and K+1 derivative outputs.
Default FCNN depth	Two-layer FCNNs for every conditioner network.	Use two hidden layers.
Default hidden units	8.	hidden_units=8.
Default RQ splines K	9.	num_splines=9.
Default training samples	2000.	n_train=2000.
Training stop	Stop when relative change between average loss over latest 50 iterations and second-latest 50 iterations is lower than 1%.	Implement rolling windows of 50 iterations; terminate if |avg_latest-avg_prev|/|avg_prev| < 0.01.
Standardization	Standardize raw samples by mean and standard deviation; orientation samples mapped to [-π,π] before standardization in	Preprocessor with inverse transform for raw-space sampling/evaluation.

	planar experiments.	
Hardware note	GPU only for neural network training; other NF-iSAM computations use CPU.	Optional GPU device for PyTorch flow training only.

Source anchor: Implementation settings and stopping rule are from Section V-A; p. 9. Standardization appears in Section IV-A practical considerations; p. 6.

10. Measurement likelihood models

$$\text{Eq. N11: relative pose: } \tilde{T}_i^j = T_i^j \exp(\xi^\wedge),\ \xi \sim \mathcal{N}(0,\Sigma),\ T_i^j := (T_i^w)^{-1} T_j^w$$

$$\text{Eq. N12: known range: } \tilde{r}_i^j = \|t_i^w - l_j^w\|_2 + \xi,\ \xi \sim \mathcal{N}(0,\sigma^2)$$

$$\text{Eq. N13: unknown association: } p(\tilde{r}_i \mid t_i^w, L_i) = \frac{1}{|L_i|} \sum_{l_j^w \in L_i} p(\tilde{r}_i \mid t_i^w, l_j^w)$$

All mixture components for unknown association are equally weighted when no prior association information is available.

Source anchor: Measurement likelihood models are specified in Section V-C; p. 9.

11. Evaluation and reproduction metrics

Metric / experiment choice	Paper usage
RMSE	Used to compare empirical mean of posterior samples with ground truth.
MMD	Used because the task is full posterior estimation; lower MMD to NSFG reference indicates a closer posterior approximation.
Reference solver	NSFG used as high-quality reference when tractable.
Unified ordering	Eliminate poses along robot trajectory first, then landmarks.
Large-scale limitation	For 416-D Plaza-like problem, reference sampling generally unavailable; RMSE only.

Source anchor: Metrics, ordering, and datasets are discussed in Sections V-C and VI; pp. 9-16.

12. Required software architecture

Module	Responsibilities
FactorGraph	Store factors, variable scopes, measurements, data association factors.
BayesTree	Build and update Bayes tree from factor graph and ordering; extract changed subtree T_Δ.
Clique	Store F_C, S_C, z_C, child separator factors, conditional sampler, separator density.
TrainingSampleSimulator	Algorithm N1; simulate intermediate samples from priors, binary factors, multimodal loop-closing factors.
FlowModel	Autoregressive RQ spline flow with density evaluation and inverse sampling.
ConditionalSamplerTrainer	Algorithm N2; train $\tilde{T}$, fix observations, partition T_S/T_F, return samplers.
NFISAMRunner	Algorithm N3; update graph, train changed subtree, root-to-leaf sample full posterior.
Preprocessor	Standardization, angle wrapping, inverse transformation to raw sample space.
Metrics	RMSE and MMD over joint/marginal samples; runtime decomposition.

13. Minimum tests for author-faithful reproduction

Test	Pass condition
Flow density test	p(x;T) evaluates by Eq. N4 and sampling by inverse T reproduces training density in simple 1-D case.
Conditional extraction test	Partitioning T(S,F) yields sampler $p(F \mid S)$ and sampler/density p(S).
Observation fixing test	$\tilde{T}(O,S,F)$ becomes T(S,F) after fixing O=o.
Algorithm 1 queue test	Binary factors are processed only when enough adjacent samples exist; otherwise pushed back.
Loop-closing test	Multimodal data association factors are treated as loop-closing factors for simulated measurement variables.
Incremental reuse test	Only changed-subtree cliques are retrained; unchanged cliques retain previous flows.
Posterior sampling test	Root-to-leaf traversal uses parent separator samples before sampling child frontal variables.
Training stop test	Training terminates when the relative change of average loss over two consecutive 50-iteration windows is below 1%.

14. Source references

- Huang, Q.; Pu, C.; Khosoussi, K.; Rosen, D. M.; Fourie, D.; How, J. P.; Leonard, J. J. “Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows,” IEEE Transactions on Robotics, 2023.

- Key source anchors used: Eq. (2)-(3) for Bayes tree and clique density; Eq. (4)-(17) for triangular normalizing flows and training; Eq. (18)-(20) and Algorithms 1-3 for sampler construction and incremental NF-iSAM; Section V-A for implementation settings; Section V-C for measurement likelihoods; Sections VI-VII for evaluation and limitations.

15. Detailed implementation pipeline for an AI coding agent

The following build order follows the authors' Algorithms 1-3. It prevents the common mistake of directly training one large flow for the whole SLAM posterior.

```
Recommended build order:
1. Implement factor graph and Bayes tree conversion/update.
2. Implement Clique objects with F_C, S_C, z_C, child clique list, and parent reference.
3. Implement Algorithm N1: TrainingSampleSimulator for clique-local intermediate samples.
4. Implement autoregressive RQ spline normalizing flow with density evaluation and inverse sampling.
5. Implement Algorithm N2: train T_tilde on ordered samples (O,S,F), fix O=o, partition into T_S and T_F.
6. Implement Algorithm N3: update graph, extract changed subtree, retrain only changed cliques, append separator
densities upward.
7. Implement root-to-leaf joint posterior sampling using cached conditional samplers.
8. Implement metrics and reproduction profiles.
```

16. Source-faithful flow training loop

```
Training loop for one clique:
Input: standardized samples X of dimension D ordered as (O,S,F)
Initialize RQ autoregressive flow T_tilde with K=9 splines and two-layer FCNN conditioners with 8 hidden units by
default.
for iteration = 1,2,...:
    compute loss L = mean_k sum_d [0.5*T_d(x^(k))^2 - log(dT_d/dx_d at x^(k))]
    update parameters by stochastic optimization
    every iteration after at least 100 iterations:
        avg_latest = mean(loss over latest 50 iterations)
        avg_prev   = mean(loss over second latest 50 iterations)
        if abs(avg_latest-avg_prev)/abs(avg_prev) < 0.01:
            stop training
Return trained T_tilde.
```

Optimizer not fully specified

The paper says the flows are constructed and trained using PyTorch and gives the loss and stopping criterion. It does not specify optimizer, learning rate, batch size, or initialization. Treat those as configurable implementation choices, while preserving the loss and stopping rule.

Source anchor: Loss, RQ splines, two-layer FCNN, hidden units=8, K=9, samples=2000, PyTorch, and 1% rolling-loss stopping rule are from Section V-A; p. 9.

17. Exact data ordering and partitioning rules

The ordering of variables inside the flow is not cosmetic. It is what lets the triangular map expose the separator density and conditional sampler.

Eq. N14: training order = (O_C, S_C, F_C)

Eq. N15: after fixing O_C=o_C, desired order = (S_C, F_C)

Block	Role before fixing observations	Role after fixing observations
O_C	Unobserved measurement variables introduced by reverting selected loop-closing likelihood factors to Bayes nets.	Fixed to measured value o_C; no longer random in desired flow.
S_C	Separator variables in the clique.	Modeled by T_S as p(S_C|z_C); passed upward as separator density.
F_C	Frontal variables in the clique.	Modeled conditionally by T_F as p(F_C|S_C,z_C); cached for posterior sampling.

18. Runtime accounting and profiling

Runtime component	Expected behavior according to paper	Implementation instrumentation
Training sample generation	Part of changed-subtree update; depends on clique dimensions and training sample count.	Timer around Algorithm N1 per clique.
Normalizing-flow training	Dominant cost; uses GPU in authors'	Timer around Algorithm N2; log iterations and

	experiments.	final loss.
Posterior sampling	Fast compared to training, but visits all cliques and grows with joint posterior dimension.	Timer around root-to-leaf traversal; report sample count and dimension.
Unchanged cliques	No retraining; reused directly.	Count reused cliques and skipped training time.

Source anchor: The paper profiles sample generation, training, and posterior sampling; it notes posterior sampling traverses all cliques and grows with dimensionality; p. 16.

19. Reproduction profiles

Profile	Use	Settings to implement
Small range-only SLAM	Demonstrate non-Gaussian posterior with/without data association ambiguity.	2-D, robot X0-X5, odometry and range measurements to landmarks; ambiguous range from X1-X4 can be associated with all detected landmarks equally.
Lawnmower 4x4 grid	Medium-scale parameter study.	Use noise/data-association settings from paper; compare RMSE/MMD/runtime and vary hyperparameters.
Simulated Plaza1	Large-scale scalability.	136 poses, 4 landmarks, 135 odometry, 136 range, 59 ambiguous measurements; final latent dimension 416-D; reference sampling generally unavailable.
Real Plaza1/Plaza2	Real dataset evaluation.	Calibrate range bias by least-squares affine relation; subtract predicted bias; use Gaussian calibrated distance error unless robust noise model is substituted.

Source anchor: Dataset setup and measurement models appear in Sections V-C and VI; pp. 9-16.

20. Implementation assumptions and open choices

Item	Paper specifies?	Recommended implementation treatment
Optimizer and learning rate	No.	Use Adam as default but expose optimizer/lr in config; record as non-paper choice.
Batch size	No.	Default to full batch for small demos or mini-batch for large cliques; expose in config.
Bayes-tree construction library	Conceptual algorithm referenced, not code API.	Use GTSAM-style Bayes tree or custom symbolic Bayes tree; preserve clique/separator semantics.
Normalizing-flow implementation details	RQ spline architecture specified; exact code not in paper.	Use standard autoregressive RQ spline implementation consistent with RQ-NSF (AR).
MMD kernel for evaluation	MMD mentioned; kernel specifics not central.	Implement standard RBF MMD and expose bandwidth.